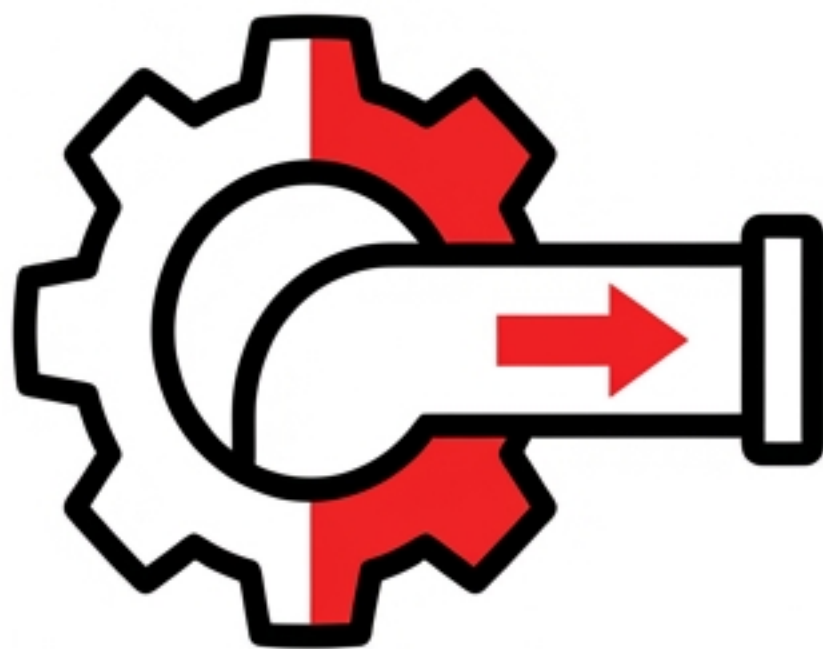




Session 8.2: ML Pipeline & scikit-learn Introduction

Your First Machine Learning Model

The \$881 Million Mistake

**\$881
MILLION**



- Zillow Offers launched a program to buy homes directly using their "Zestimate" machine learning model.
- Result: The model failed to account for market shifts. Zillow lost \$881M, shut down the entire program, and laid off 2,000 employees (25% of their workforce).

Building a model is easy. Trusting it requires pipeline discipline.

The root cause was inadequate evaluation on out-of-sample data.

Session Objectives



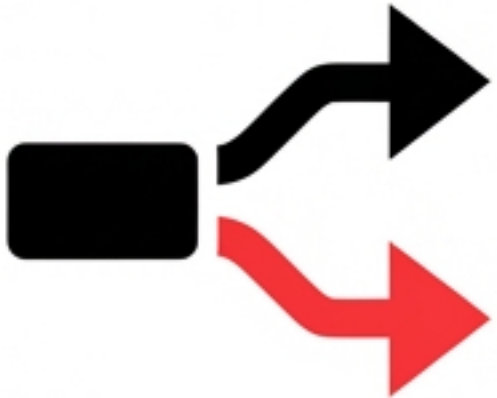
1. Distinguish ML Paradigms

Understand Supervised vs. Unsupervised learning.



2. Data Preprocessing

Handle messy, real-world data before it reaches the model.



3. The Golden Rule

Split data into separate Train and Test sets to ensure true evaluation.



4. scikit-learn API

Prepare the architecture for your first ML model.

Supervised vs. Unsupervised Learning

Supervised Learning

Data: Labeled (Predict y from X)

Goal: Map inputs to known outputs.

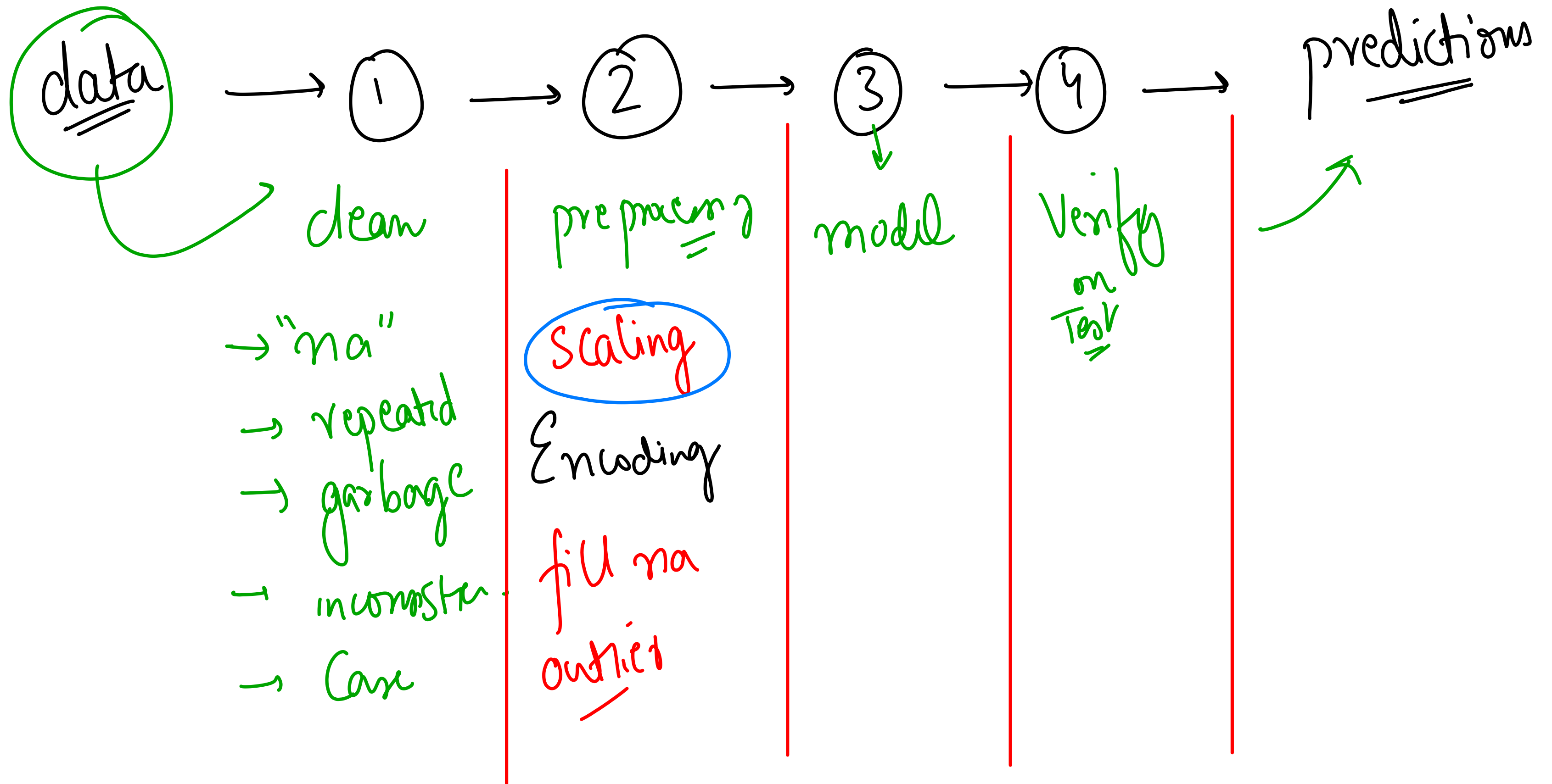
Types: Regression (continuous) & Classification (categories).

Unsupervised Learning

Data: Unlabeled (Find patterns in X only)

Goal: Discover hidden structures and groupings.

Types: Clustering & Dimensionality Reduction.



age Salary Gender.

		M	
		F	
		M	
		F	

0-100

10000-1000000

m/f

Categorical

Encoding

0

1

0

1

weights

bias

$$y = m \cdot x + c$$

$$y = m_1 \text{ age} + m_2 \text{ Salary}$$

$$+ m_3 \text{ Gender} + \text{bias}$$

$$y = m x + c$$

$$y = m_1 \overset{1}{x_1} + m_2 \overset{1}{x_2} + m_3 \overset{1}{x_3} + b$$

Coefficients

bias

$y = 10, 50, 101$

0-100

10000 - 10,00,000

[]

min max scaling

$$\begin{matrix} 0 & 0.1 & 0.3 & 0.9 & 1 \\ \swarrow & & & & \nearrow \\ [0, & 1, & 3, & 9, & 10] = \end{matrix}$$

$$\frac{x - \min}{\max - \min} = \frac{x - 0}{10 - 0}$$

Standard Scaling

$$\begin{matrix} [- & - & - & - & -] \\ [0, & 1, & 3, & 9, & 10] \end{matrix}$$

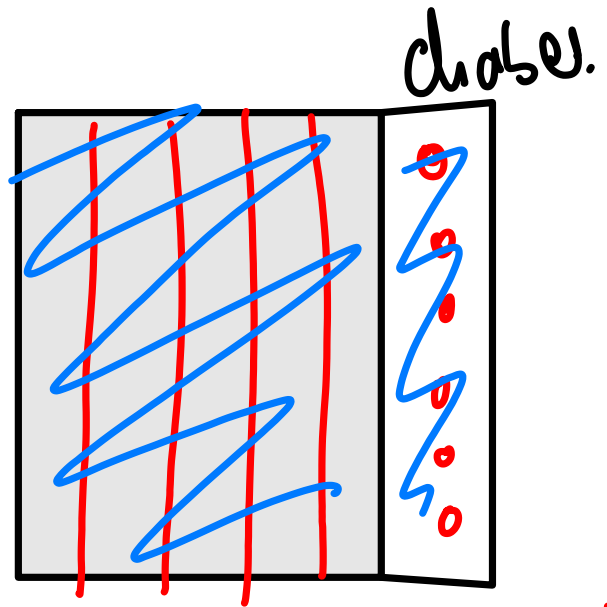
$$= \frac{(x - \mu)}{\sigma}$$

μ ← mean

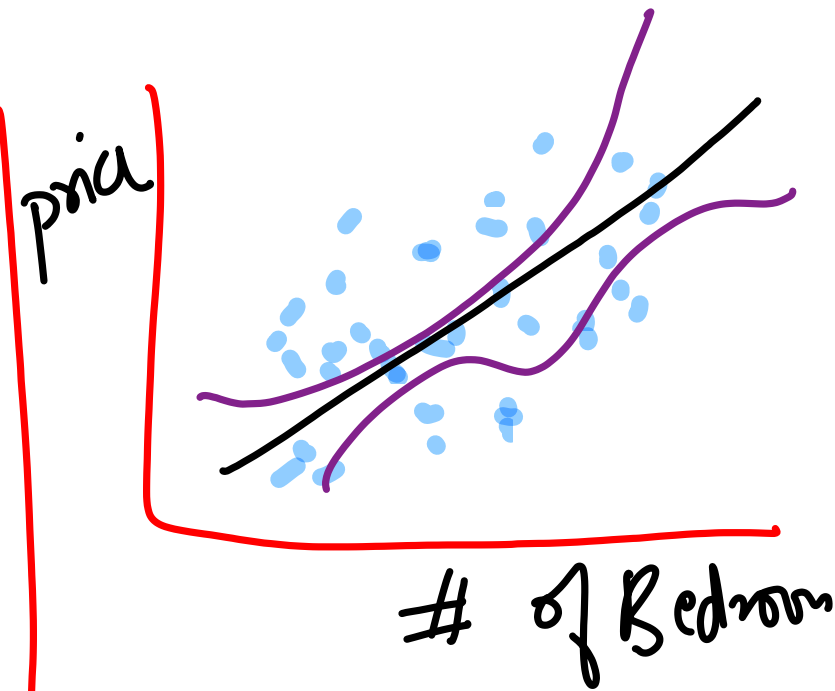
μ
 σ

$$D = \frac{0 - \mu}{\sigma}$$

Supervised



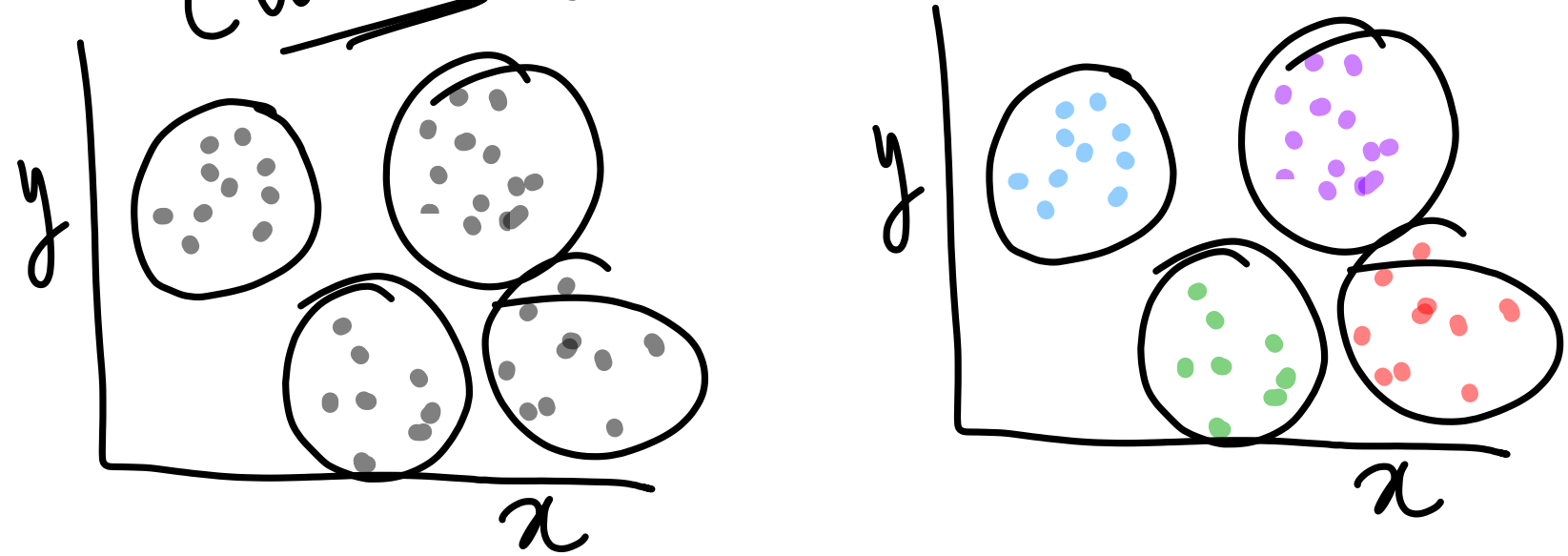
Classifier



Regressor

Unsupervised

Clustering



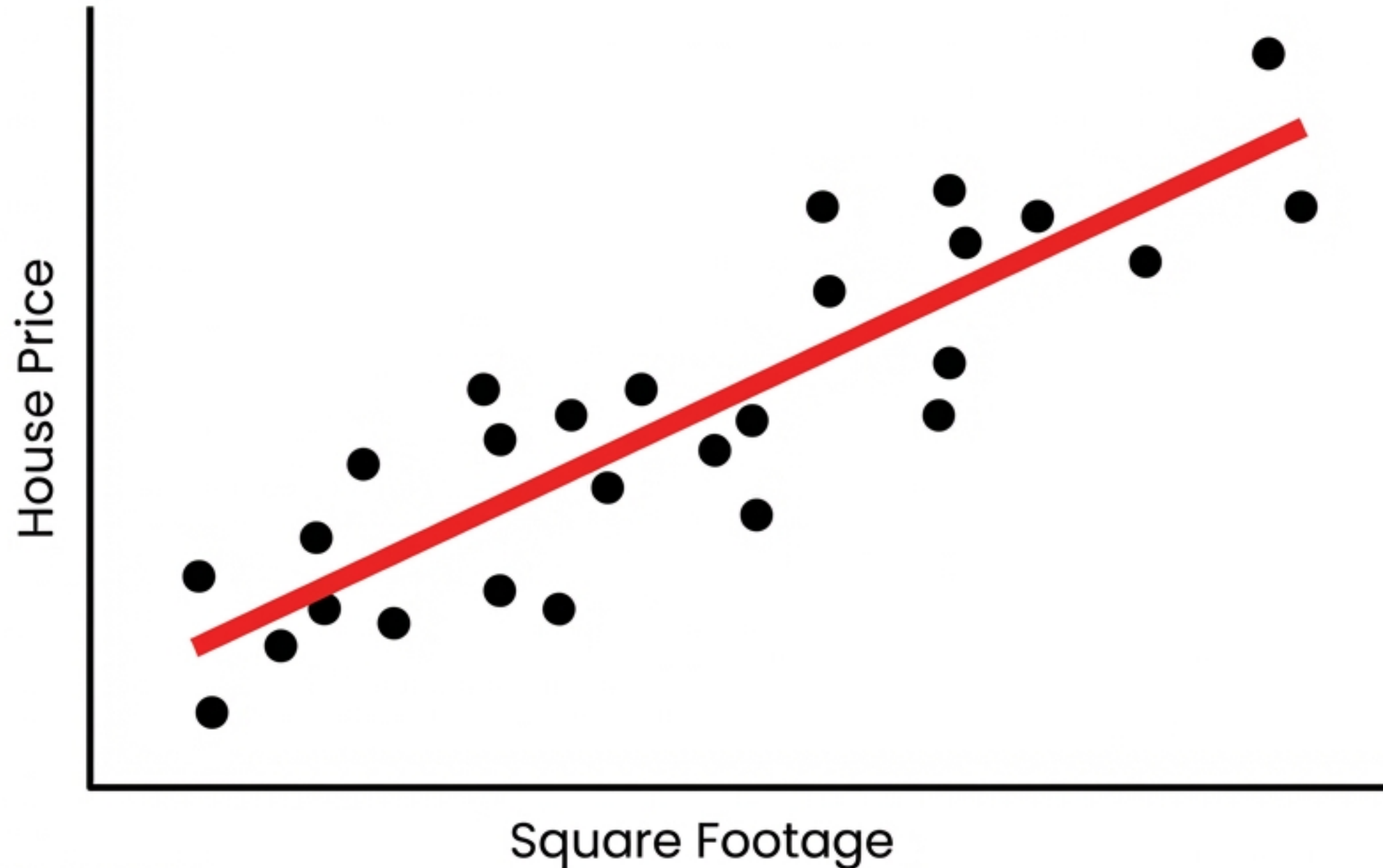
Dimensionality reduction

1000

→ 5 dimension

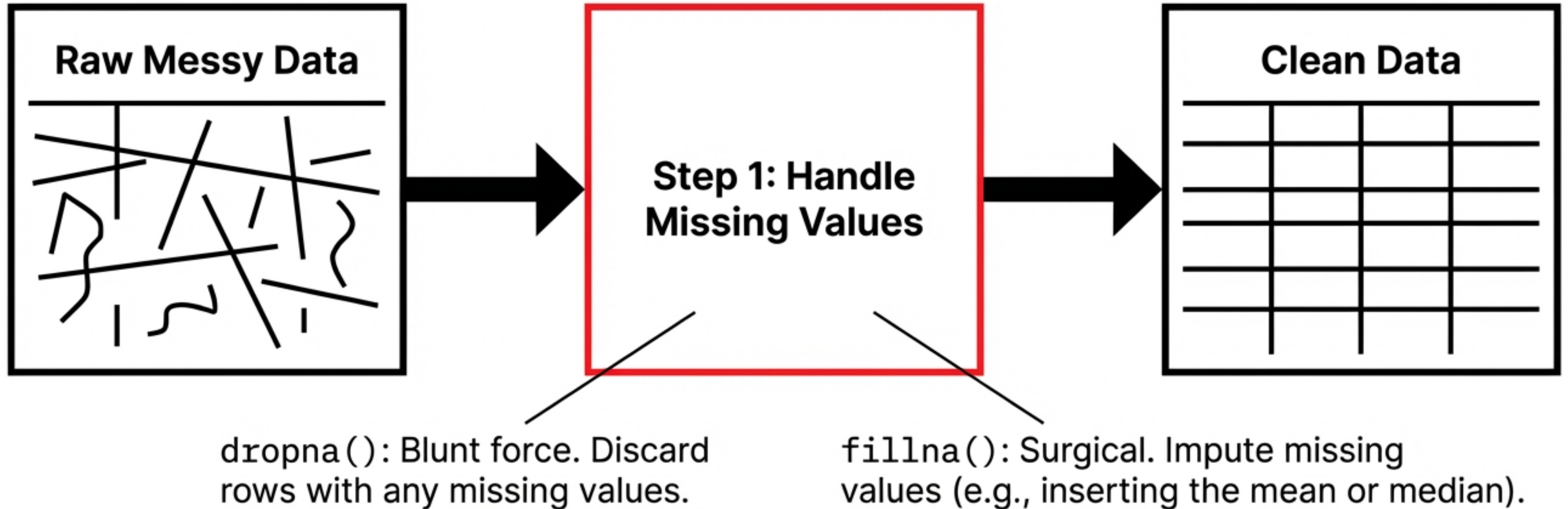
Our Focus: Supervised Regression

Predicting continuous numerical values.



Data Preprocessing: Garbage In, Garbage Out

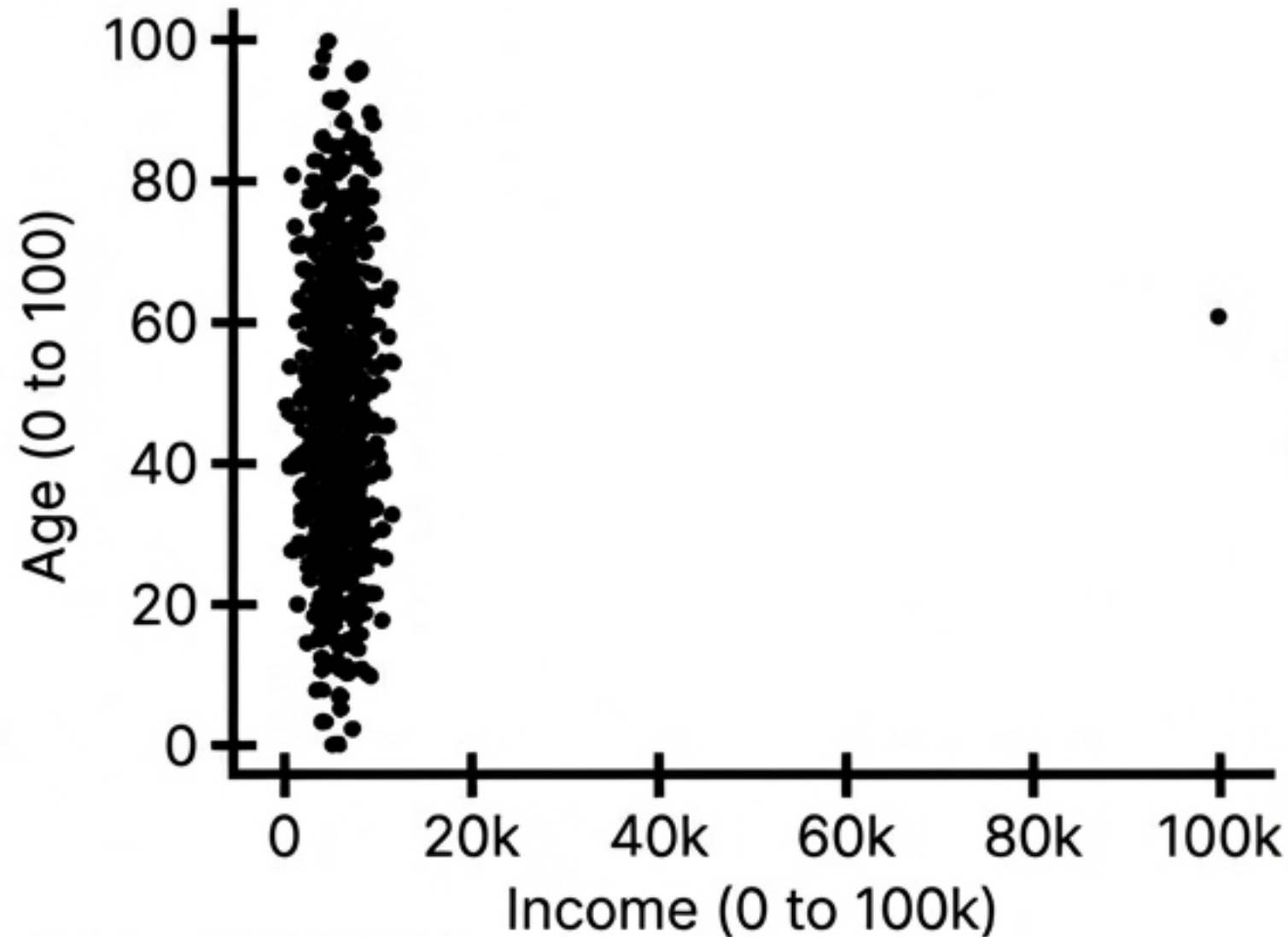
ML models are mathematical engines; they cannot compute raw, messy data with gaps.



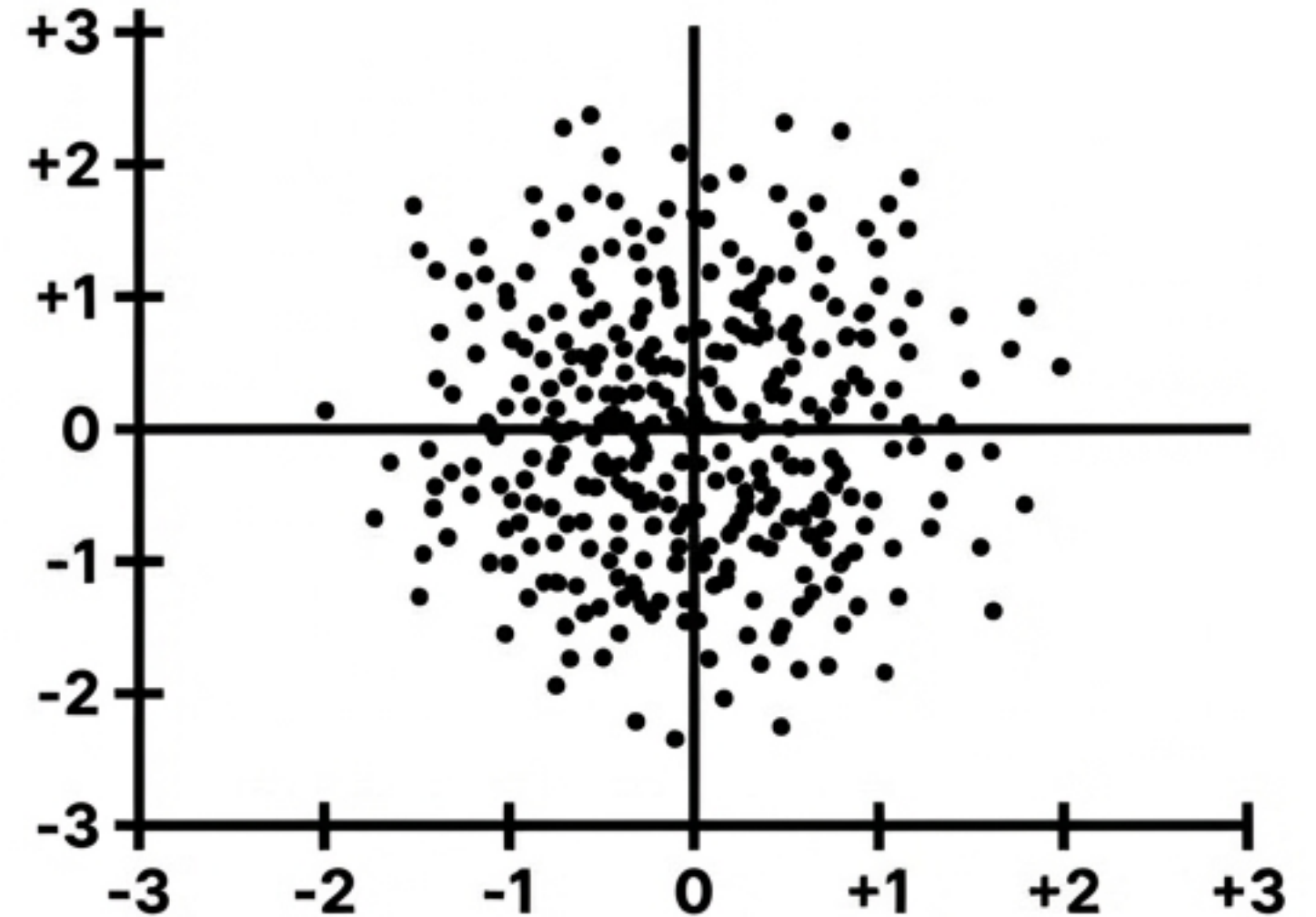
Leveling the Playing Field

Scaling is critical so astronomically large numbers (like Income: \$80,000) don't mathematically overpower smaller numbers (like Age: 35) inside the model.

BEFORE



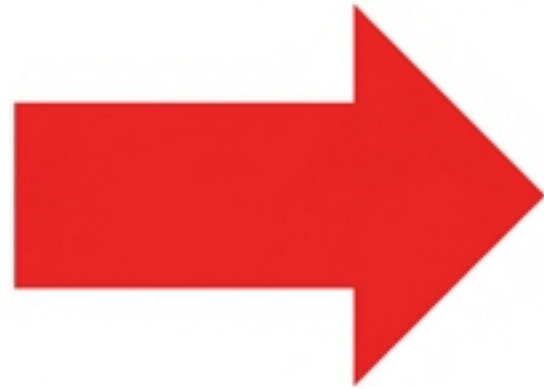
AFTER (StandardScaler)



Machines Only Read Numbers

One-hot encoding converts text categories into binary columns so the math can work.

Color
Red
Blue
Green

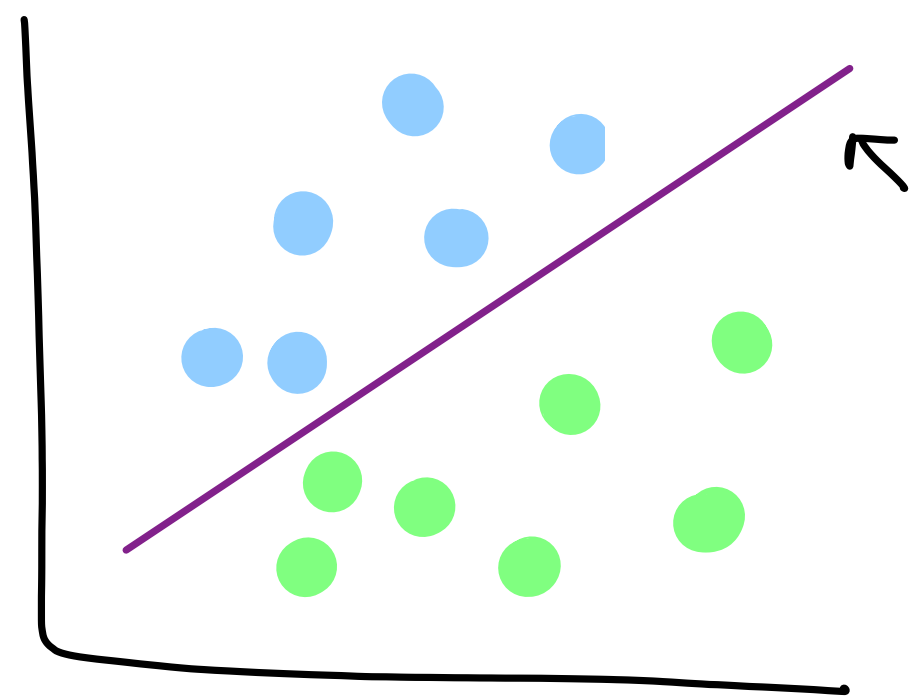
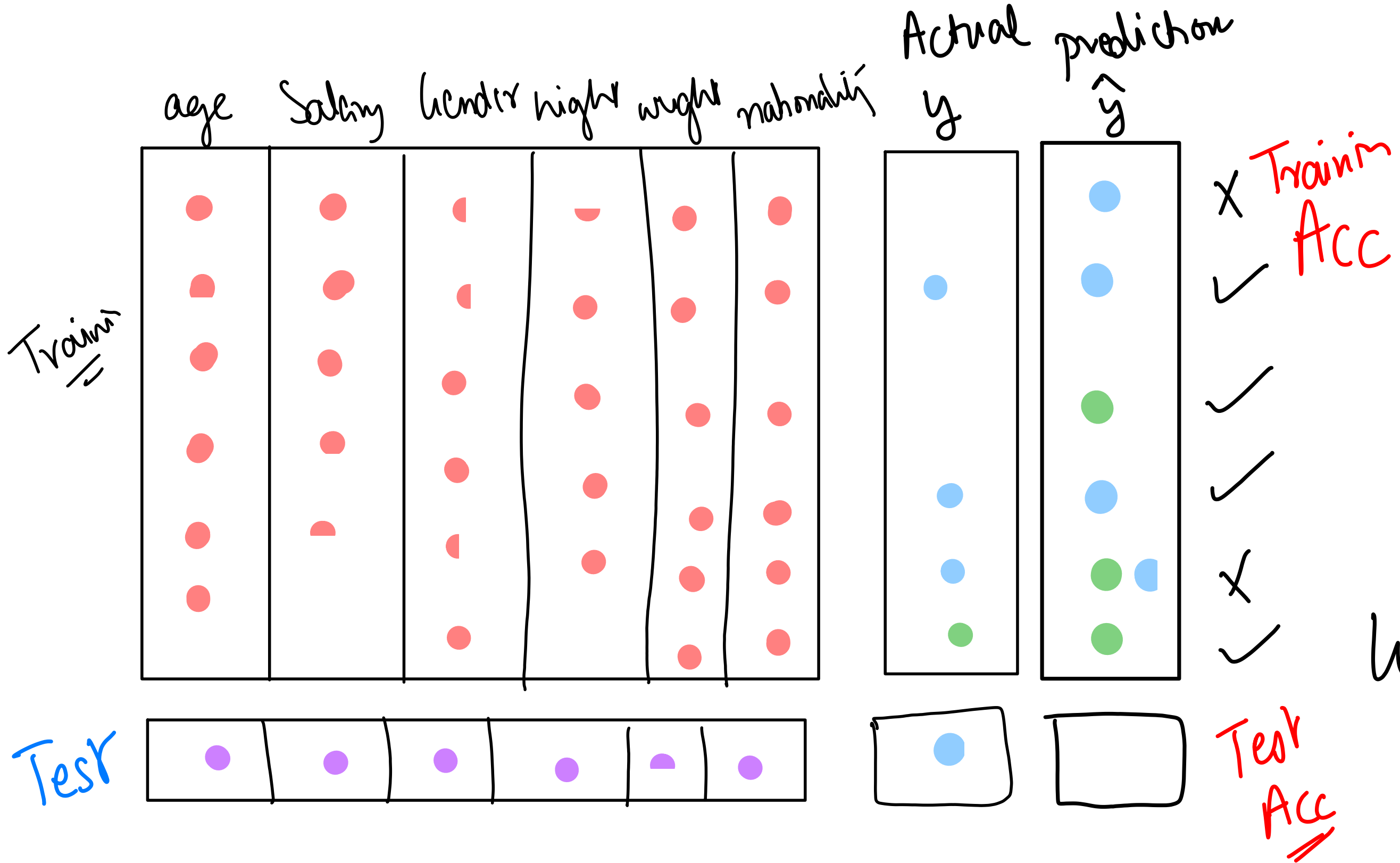


Color_Red	Color_Blue	Color_Green
1	0	0
0	1	0
0	0	1

THE GOLDEN RULE

NEVER

**evaluate a model on the same
data used to train it.**



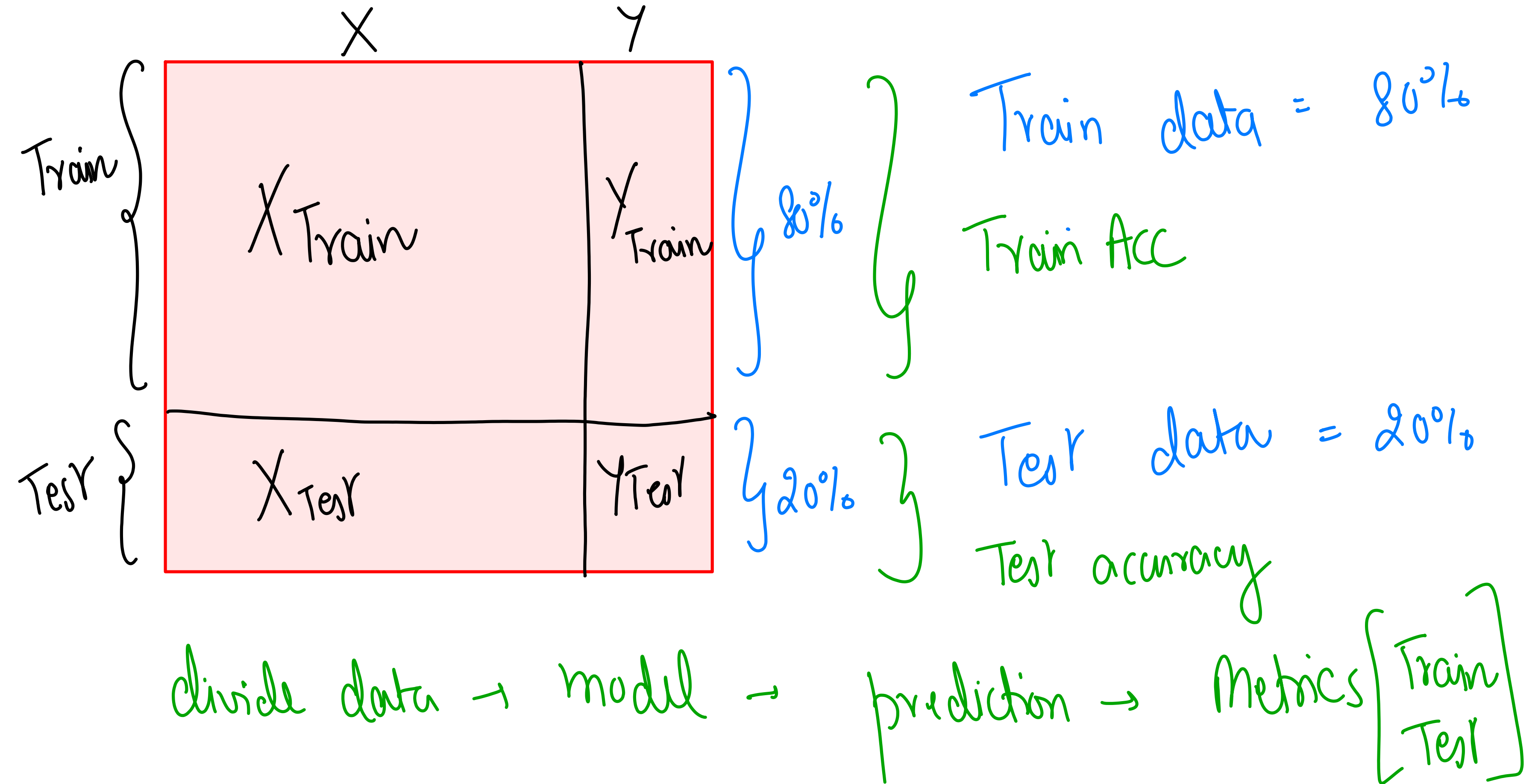
training

$$\text{loss} = \sum \frac{(y - \hat{y})^2}{n}$$

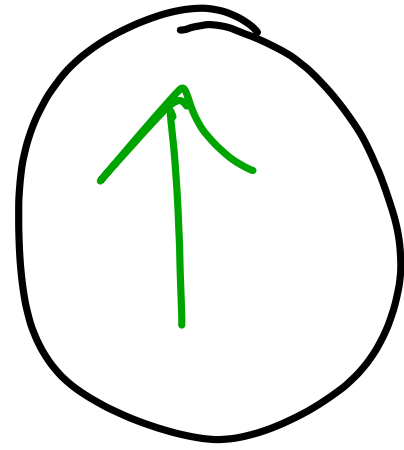
GD

$y =$

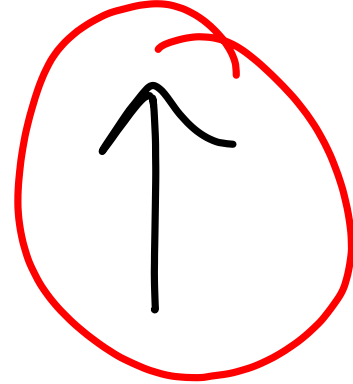
$$\frac{wx + b}{\text{weight} \quad \text{bias}}$$



Train Acc

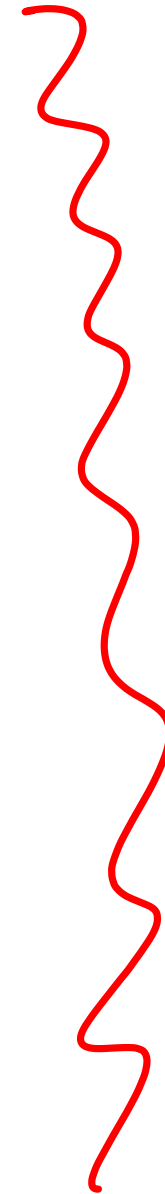


Test Acc



Overfitting

underfitting



$$y = mx + c$$

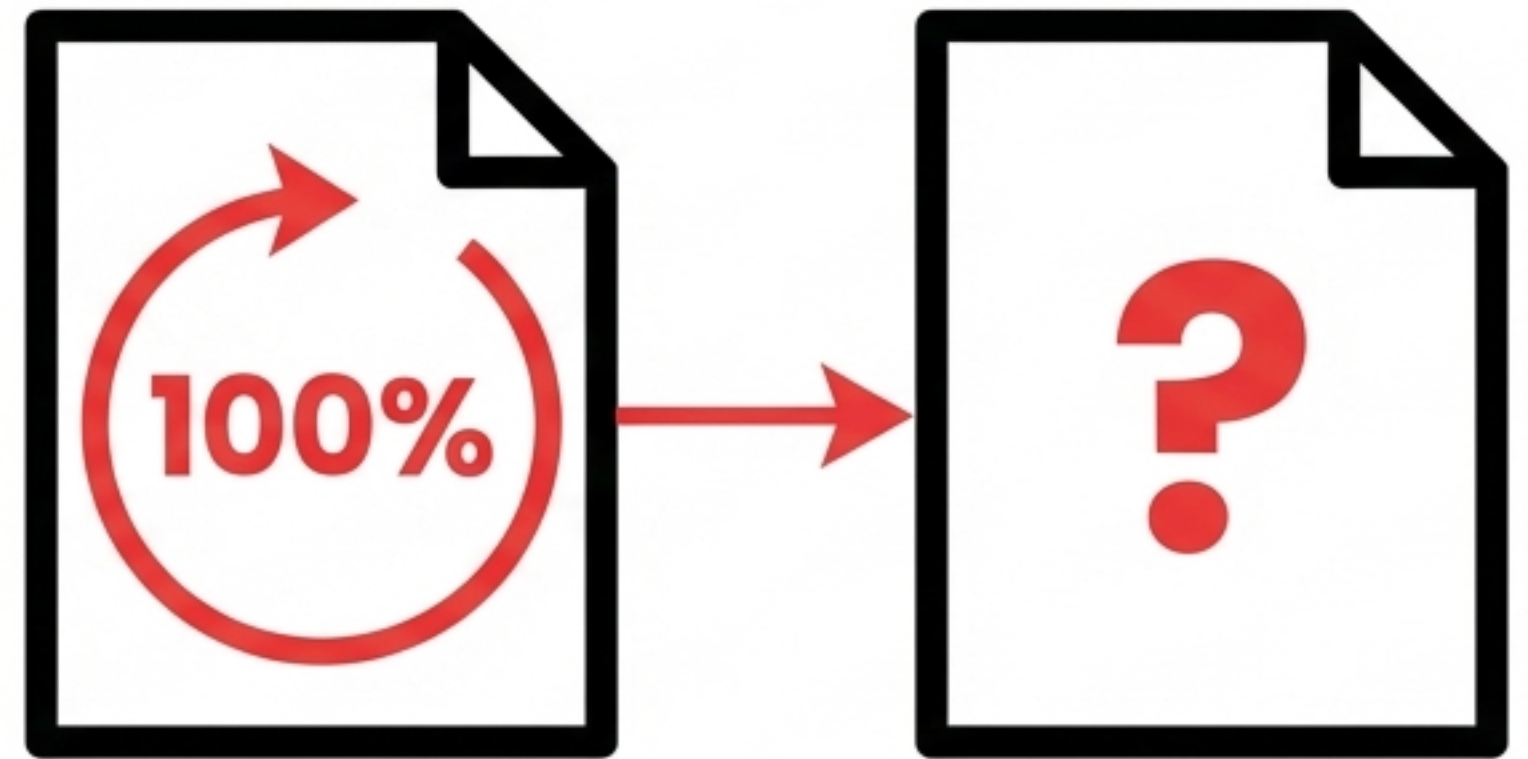
Error $\sum \frac{(y - \hat{y})^2}{n}$

start

y

Memorization vs. Learning

- **The Setup:** A student studies past exam papers. ✓
- **The Trap:** If tested on those exact same.
- **The Trap:** If tested on those exact same past papers, the student will score 100%. They didn't learn the subject; they memorized the answers. ✓
- **The Solution:** Test the student on a new, unseen exam to prove they actually learned the underlying concepts. ✓



Takeaway: The Training set is the past papers. ✓
The Test set is the unseen new exam. ✓

The 80/20 Train/Test Split



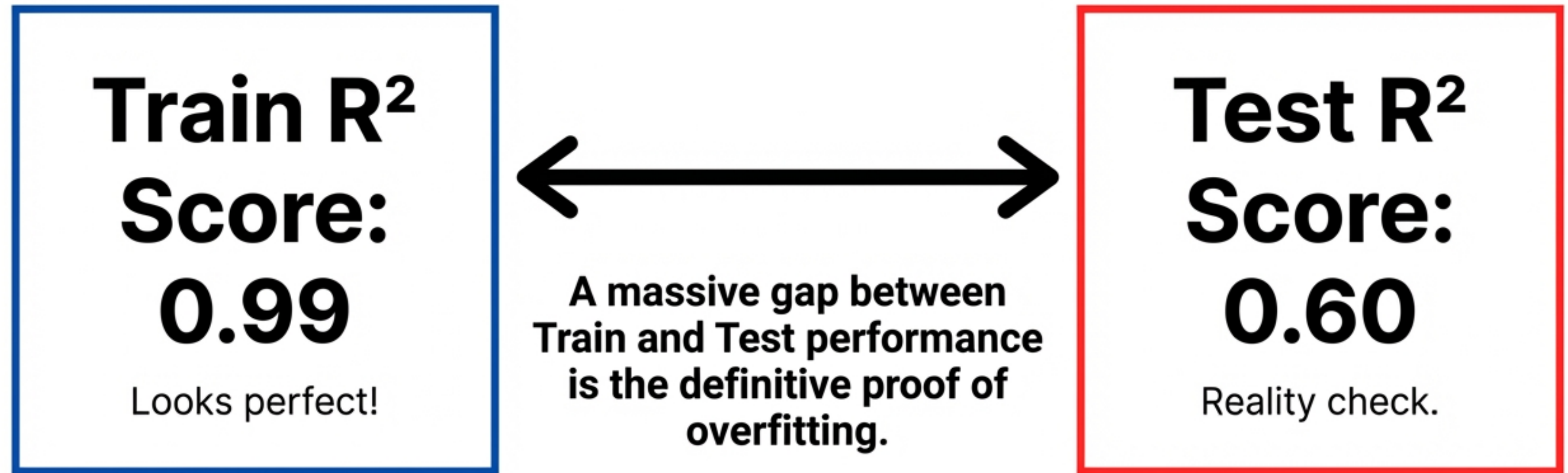
Training Data: Used to **FIT** the model.

Test Data: Used
ONLY to **EVALUATE**.

Fit on the 80% → **Evaluate on the 20%**

Spotting the Illusion of Perfect Scores

Overfitting occurs when a model memorizes the training data perfectly but fails completely on new, unseen data.

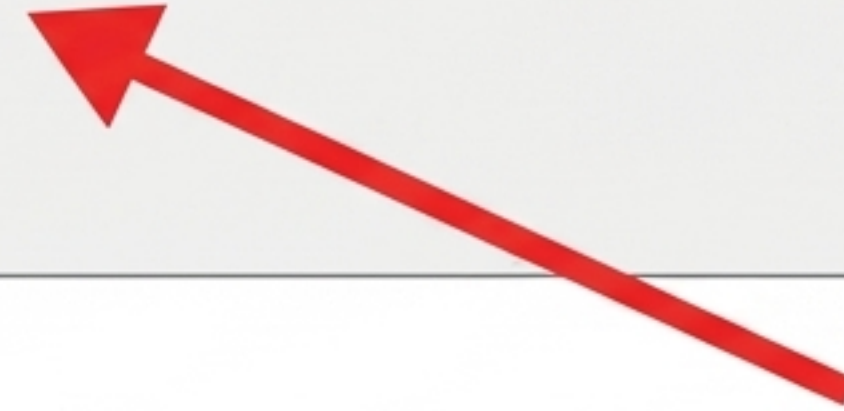


Splitting Data in scikit-learn

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
```



Reserves exactly 20% of data for the test set.



Ensures the random shuffle is reproducible every time you run it.

Pipeline Checkpoint



1. Paradigm

Supervised Regression maps labeled inputs (X) to continuous outputs (y).



2. Preparation

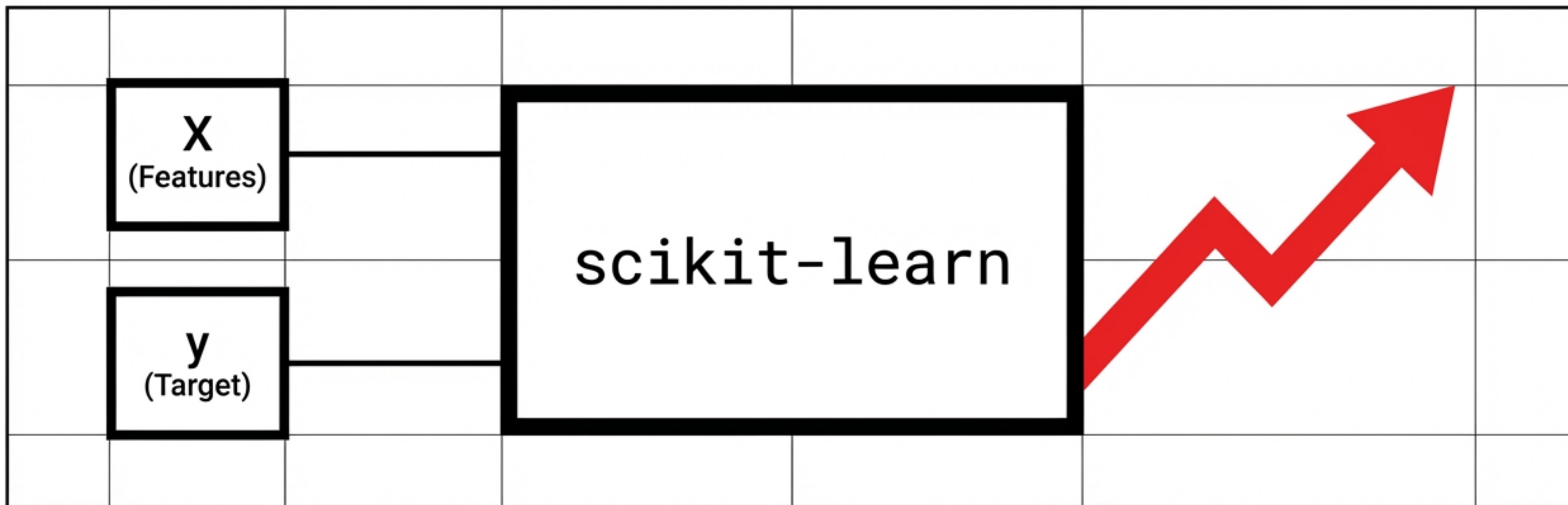
Raw data must be aggressively cleaned, scaled, and encoded before any modeling begins.



3. Discipline

The Train/Test split is the ultimate defense against overfitting and memorization.

Executing Machine Learning with the scikit-learn API

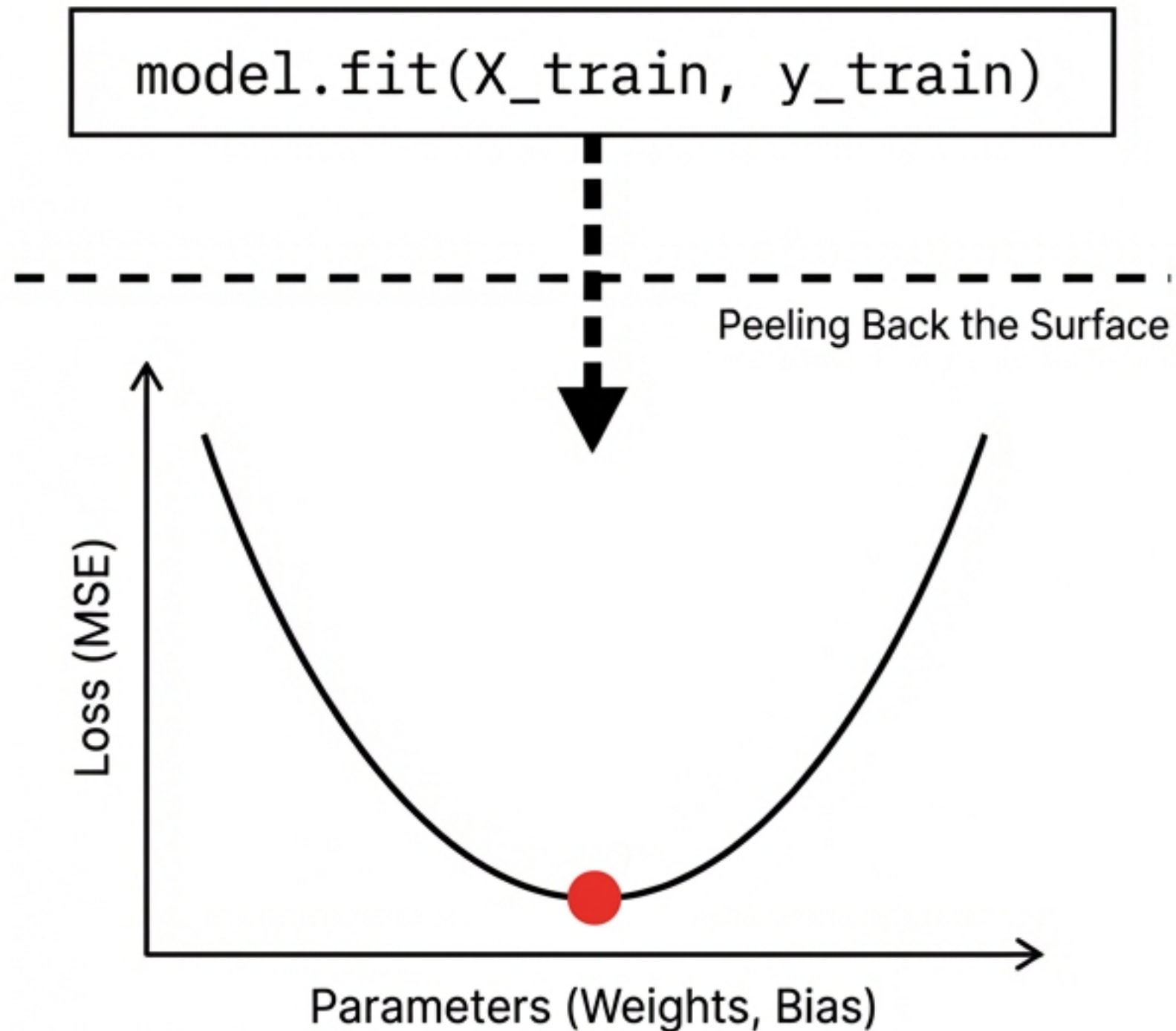


- The math, data preparation, and Python fundamentals converge here.
- One unified library handles the complete model execution process.

A Universal Three-Step Pattern for Every Model

<pre>model = LinearRegression()</pre>	Step 1: Create. Initializes the chosen algorithm with default or custom hyperparameters.
<pre>model.fit(X_train, y_train)</pre>	Step 2: Learn. The model extracts the underlying mathematical patterns strictly from the training data.
<pre>predictions = model.predict(X_test)</pre>	Step 3: Predict. The model applies its newly learned rules to unseen data to test its accuracy.

The Gradient Descent Engine Inside the **fit()** Command



Executing **.fit()** automatically triggers the mathematical optimization loop.

- **Initialize:** Starts with random parameters (weights and bias).
- **Compute:** Calculates the gradients of the Mean Squared Error (MSE) loss function.
- **Update:** Adjusts parameters using the optimized learning rate.
- **Repeat:** Loops until the minimum error is found—replacing weeks of manual math with one line of code.

Writing Your First End-to-End Machine Learning Model

```
from sklearn.linear_model import LinearRegression

# 1. Instantiate the algorithm
model = LinearRegression()

# 2. Fit to the training data (Gradient Descent runs here)
model.fit(X_train, y_train)

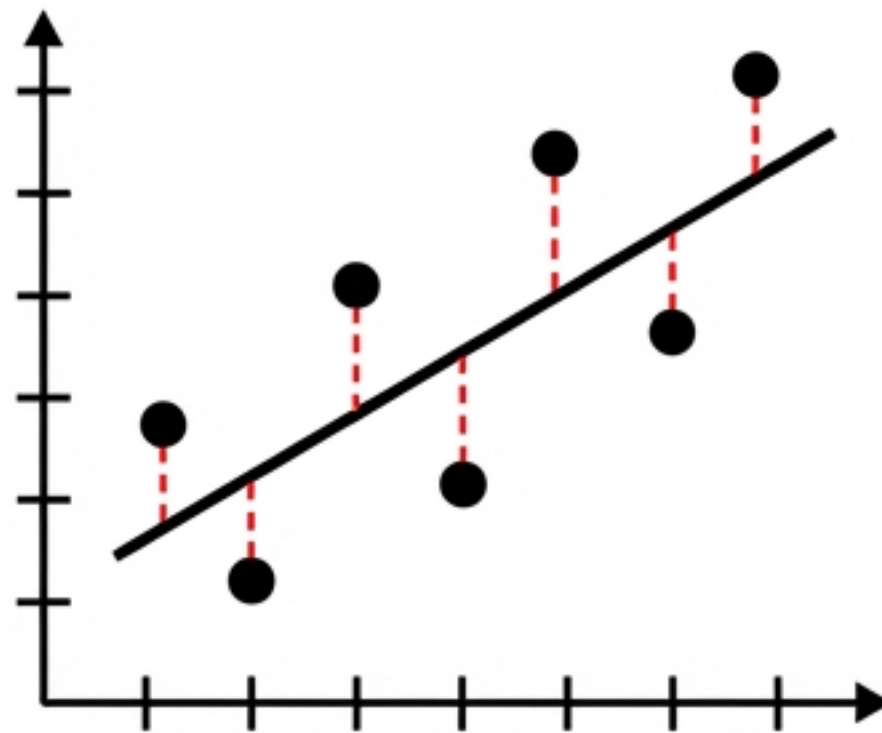
# 3. Generate predictions on unseen test data
y_pred = model.predict(X_test)

print('Optimized Coefficients:', model.coef_)
```



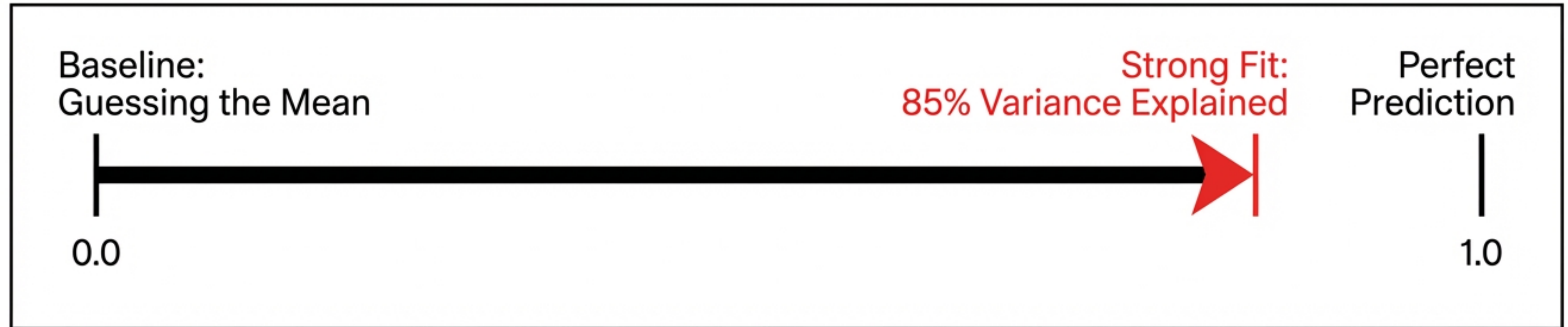
Measuring Absolute Performance with RMSE

$$\text{RMSE} = \sqrt{\frac{\sum (y - \hat{y})^2}{n}}$$



- **Definition:** The square root of the average of squared differences between predictions and actual observations.
- **The Advantage:** Measures the error in the original units of the target variable (e.g., dollars, minutes, kilograms).
- **Interpretation:** An RMSE of \$50,000 means the model's house price predictions are off by \$50k on average. Lower is better; 0 is perfect.

Measuring Relative Variance with R-Squared (R^2)



Definition: The proportion of the variance in the dependent variable that is predictable from the independent variables.

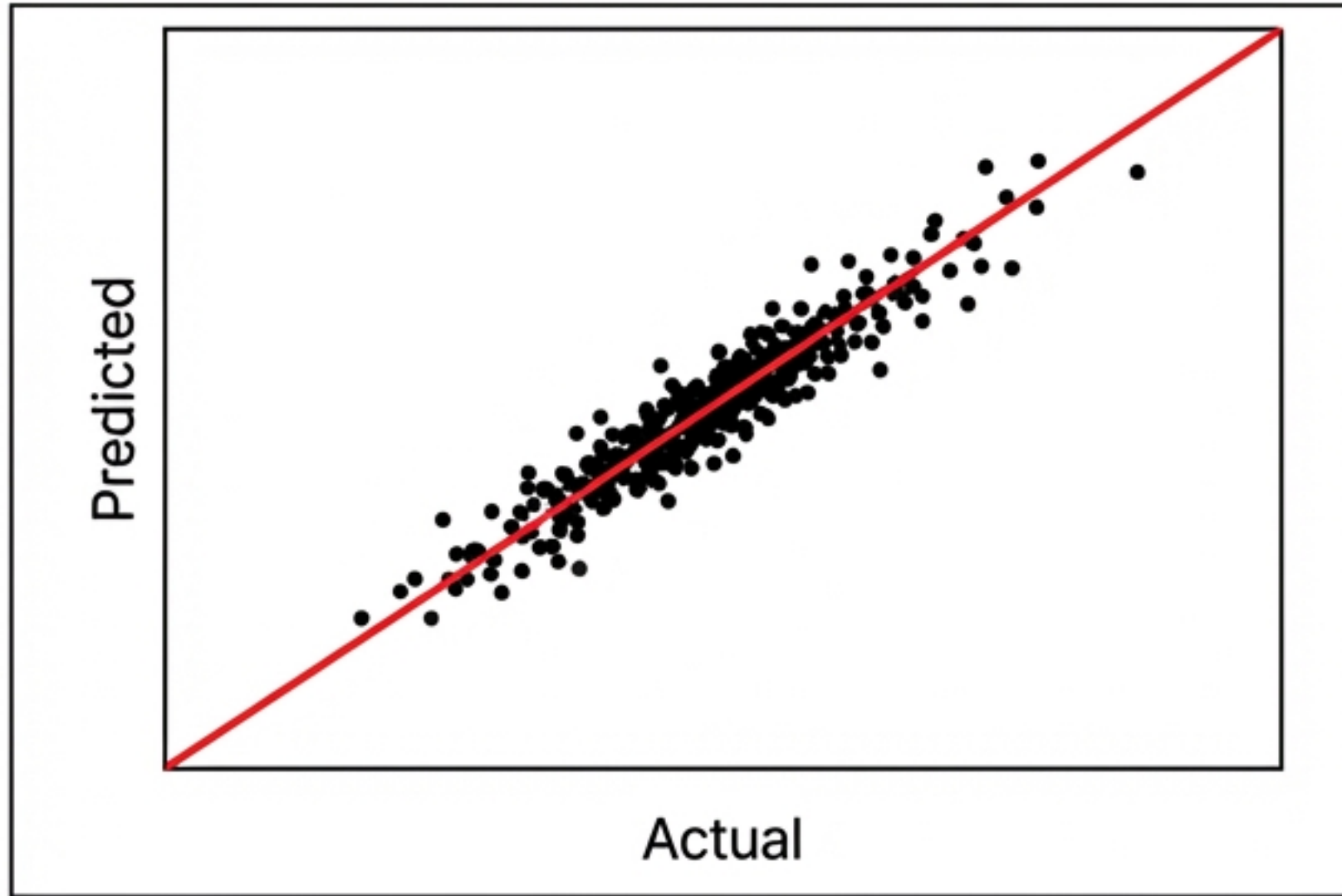
Scale Limits:

1.0: The model perfectly maps the inputs to the targets.

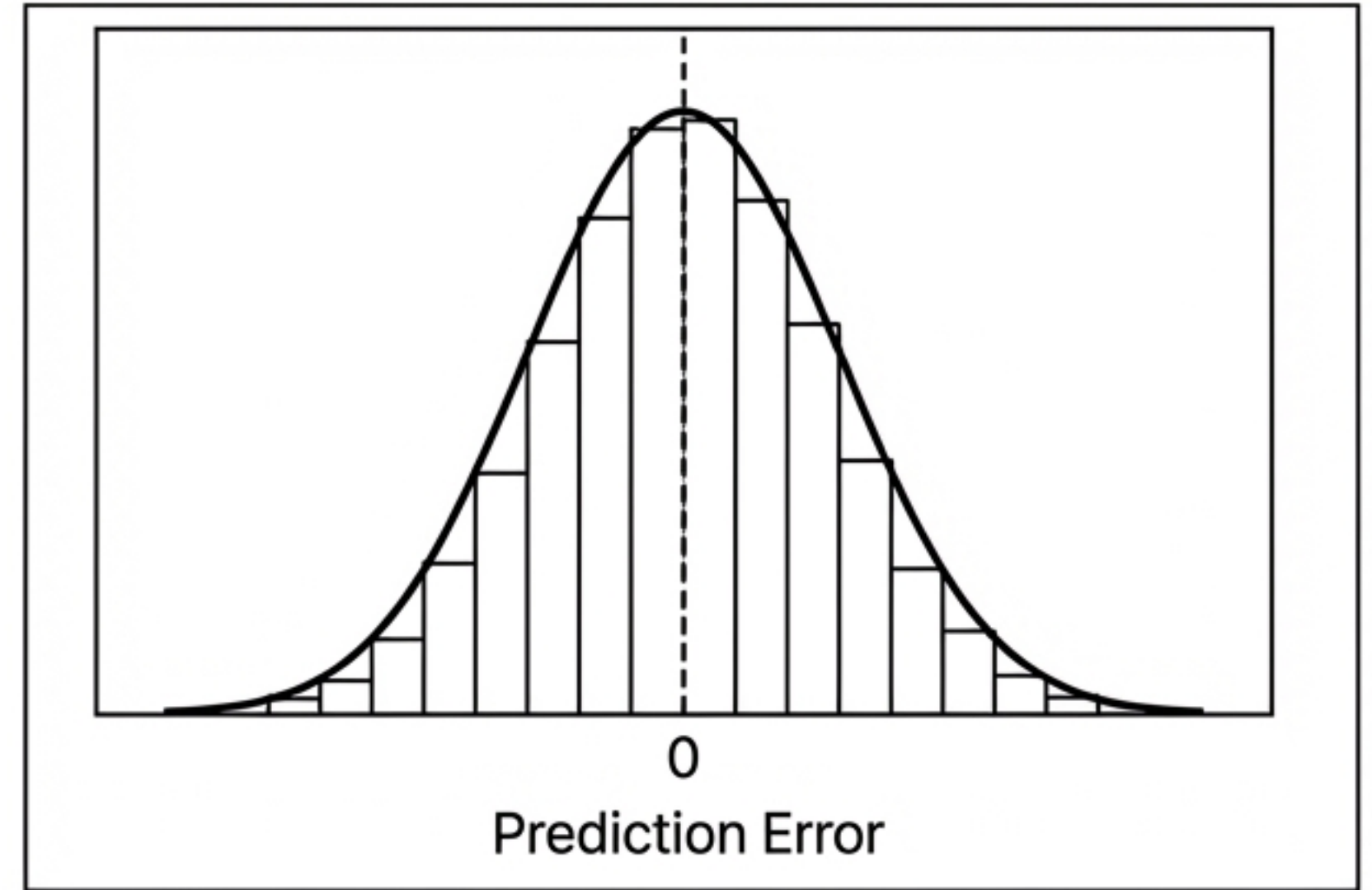
0.0: The model is mathematically no better than ignoring the features and simply guessing the average value every time.

< 0.0: The model actually performs worse than predicting the mean.

Visual Diagnostics Protect Against Hidden Model Flaws



- **The Diagonal Rule:** If actual vs. predicted points form a random cloud, the model failed to learn. Points must hug the 45-degree axis.



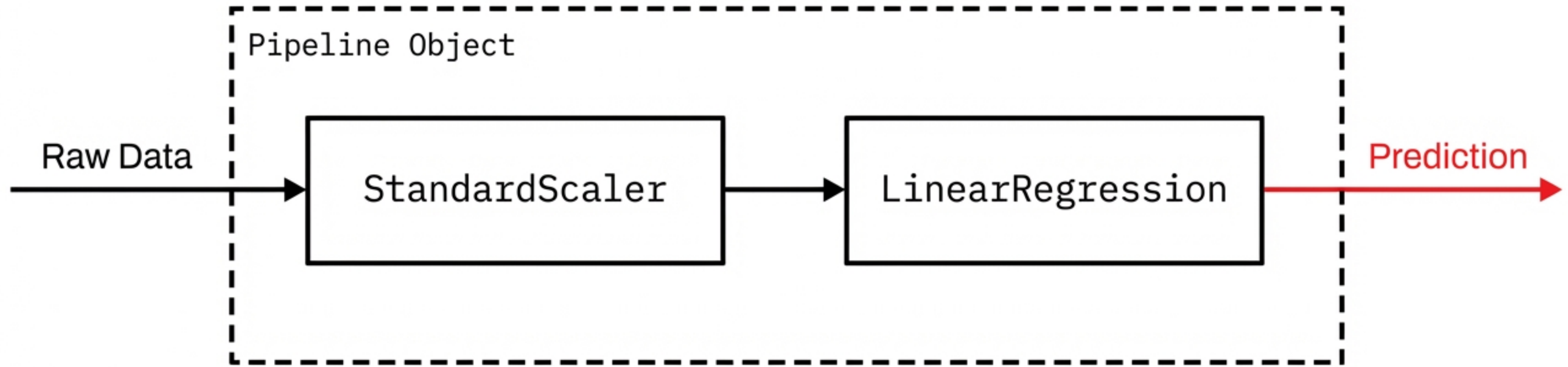
- **The Residual Rule:** Prediction errors should form a normal distribution centered at zero. Skewed residuals indicate consistent over/under-predicting.

The Overfitting Trap: Memorization vs. Generalization

Condition	Train R^2	Test R^2	Status
Healthy Model	0.85	0.82	Validated 
Memorized Data	0.99	0.55	Overfit 

- Models can learn the training data too well, capturing random noise instead of the underlying mathematical signal.
- An overfit model collapses when exposed to unseen data.
- The Diagnostic Test: If Train R^2 is significantly higher than Test R^2 , the model has memorized the answers to the practice test rather than learning the subject.

Preventing Data Leakage with the scikit-learn Pipeline



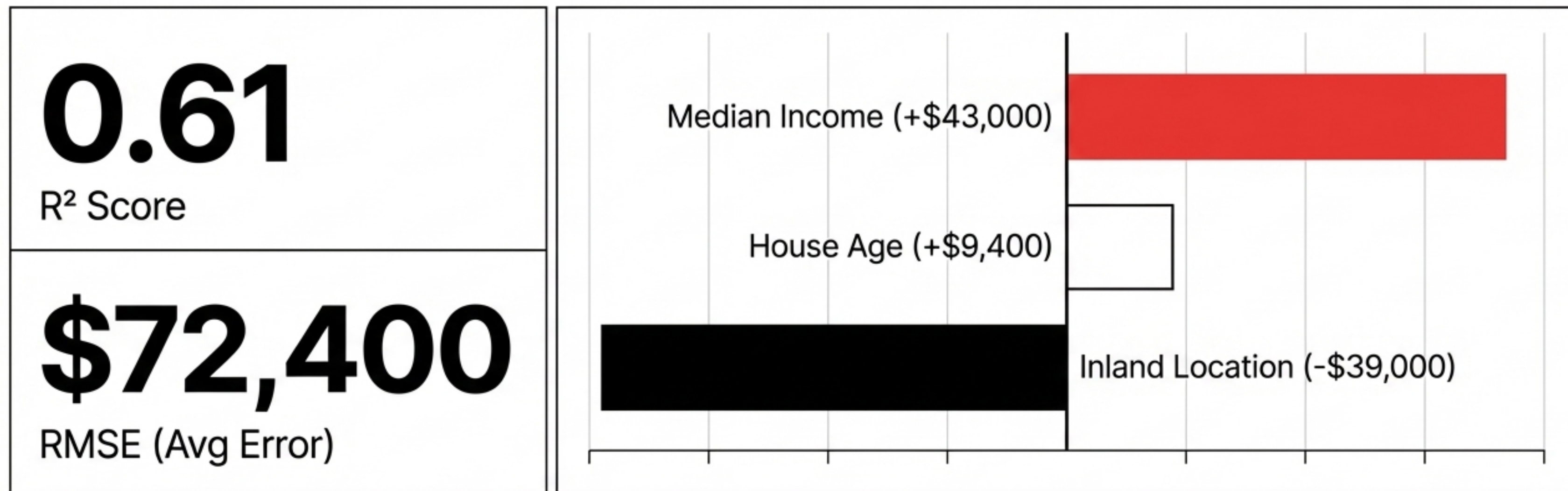
- The **Danger**: Applying preprocessing (like scaling) to the training and test sets separately invites data leakage and calculation errors.
- The **Professional Standard**: `sklearn.pipeline.Pipeline`.
- The **Mechanism**: Packages the scaler and the algorithm together. Calling `.fit()` on the pipeline ensures data is scaled and the model is trained in one seamless, mathematically isolated motion.

Mini-Project: California House Price Predictor



Executing the exact workflow used in production environments to transition from raw **tabular data** to an **evaluated** predictive model.

Model Diagnostics and Feature Impact



Performance Tracking: The linear pipeline successfully explains 61% of the total variance in California house prices, with an average prediction error of \$72,400.

Interpretability: Median income operates as the single strongest positive driver of property value, while inland geographical locations apply the strongest negative pressure.

The Multi-Million Dollar Cost of a Broken Pipeline

\$881,000,000



- Zillow relied on historical training data **without adequately evaluating model performance on volatile, out-of-sample data** during market shifts.
- The model over-predicted values, causing Zillow Offers to systematically overpay for inventory.
- The Fallout: **Nearly a billion dollars lost and 2,000 jobs eliminated.**
- The Core Lesson: Gradient Descent rarely fails. **The evaluation pipeline is where companies lose millions. Trust the pipeline, not the hype.**

Synthesis Part 1: Data and Paradigms



Paradigms

Supervised learning maps features (X) to known targets (y). Unsupervised learning extracts hidden structures from unlabeled data.



Preprocessing Mandate

Algorithms require numerical, identically scaled data. `StandardScaler` prevents larger numbers from dominating the loss function.



The Golden Rule

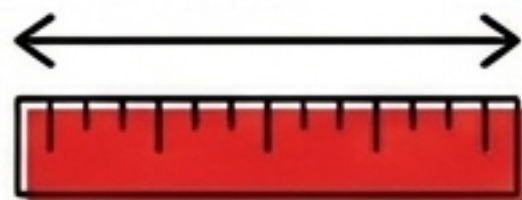
Never evaluate a model on the data used to train it. An 80/20 Train/Test split is an absolute requirement for validation.

Synthesis Part 2: Execution and Evaluation



The API

Execution is standardized.
`.fit(X, y)` runs the optimization loop;
`.predict(X)` applies the learned parameters to new data.



Dual Metrics

RMSE grounds error in absolute, business-relevant units. R^2 measures the relative proportion of variance successfully explained.



The Memorization Check

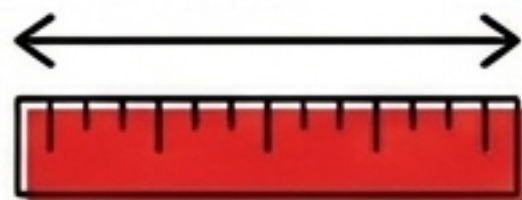
Continuously compare Train R^2 against Test R^2 . A significant drop on the test set is the primary indicator of catastrophic overfitting.

Synthesis Part 2: Execution and Evaluation



The API

Execution is standardized.
`.fit(X, y)` runs the optimization loop;
`.predict(X)` applies the learned parameters to new data.



Dual Metrics

RMSE grounds error in absolute, business-relevant units. R^2 measures the relative proportion of variance successfully explained.



The Memorization Check

Continuously compare Train R^2 against Test R^2 . A significant drop on the test set is the primary indicator of catastrophic overfitting.

10000
10001

Join at menti.com | use code 1803 1090

Mentimeter

Predicting house prices from features (sqft, bedrooms, age) is an example of:

12000

35 ✓



Supervised regression

10 ✗



Unsupervised learning

16 ✗



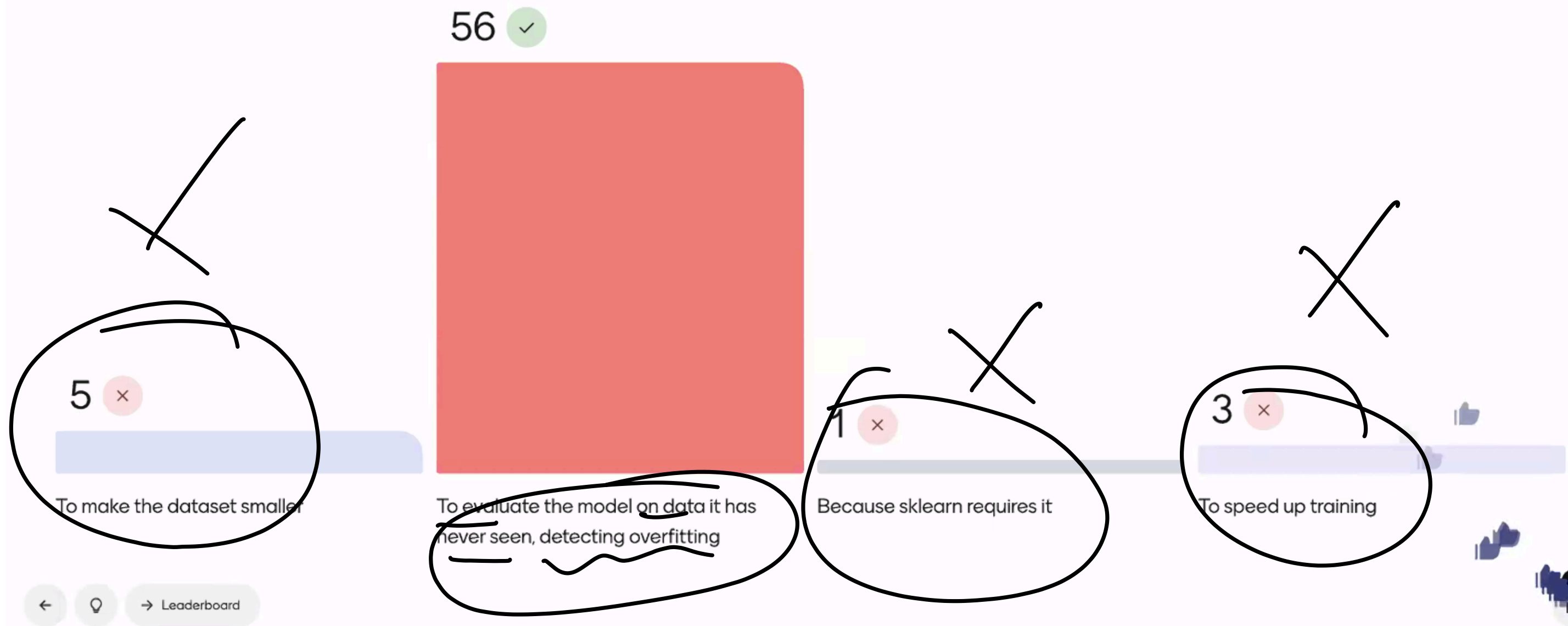
Supervised classification

2 ✗

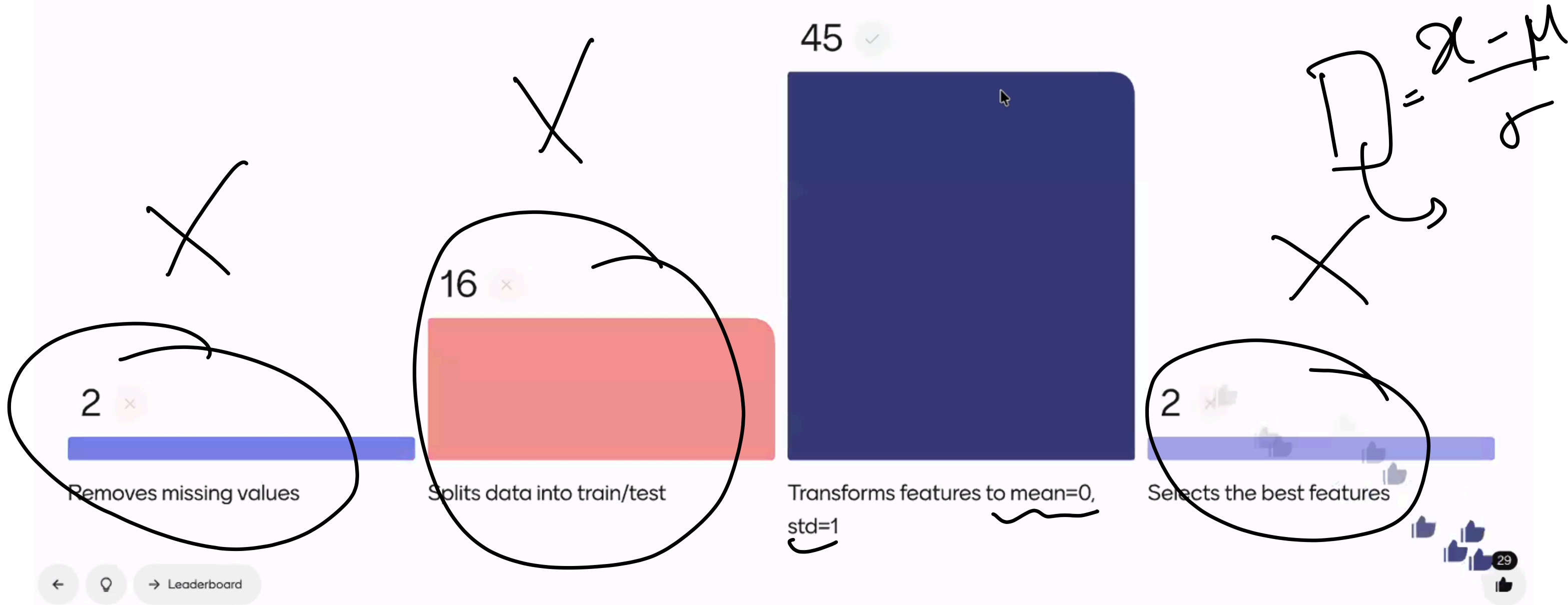


Clustering

Why do we split data into train and test sets?



What does StandardScaler do?



In train_test_split(X, y, test_size=0.2), the test set contains:

46 ✓



20% of the data

8 ✗



80% of the data

7 ✗



All the data

1 ✗



Only the target variable

Train

→ Leaderboard



28



